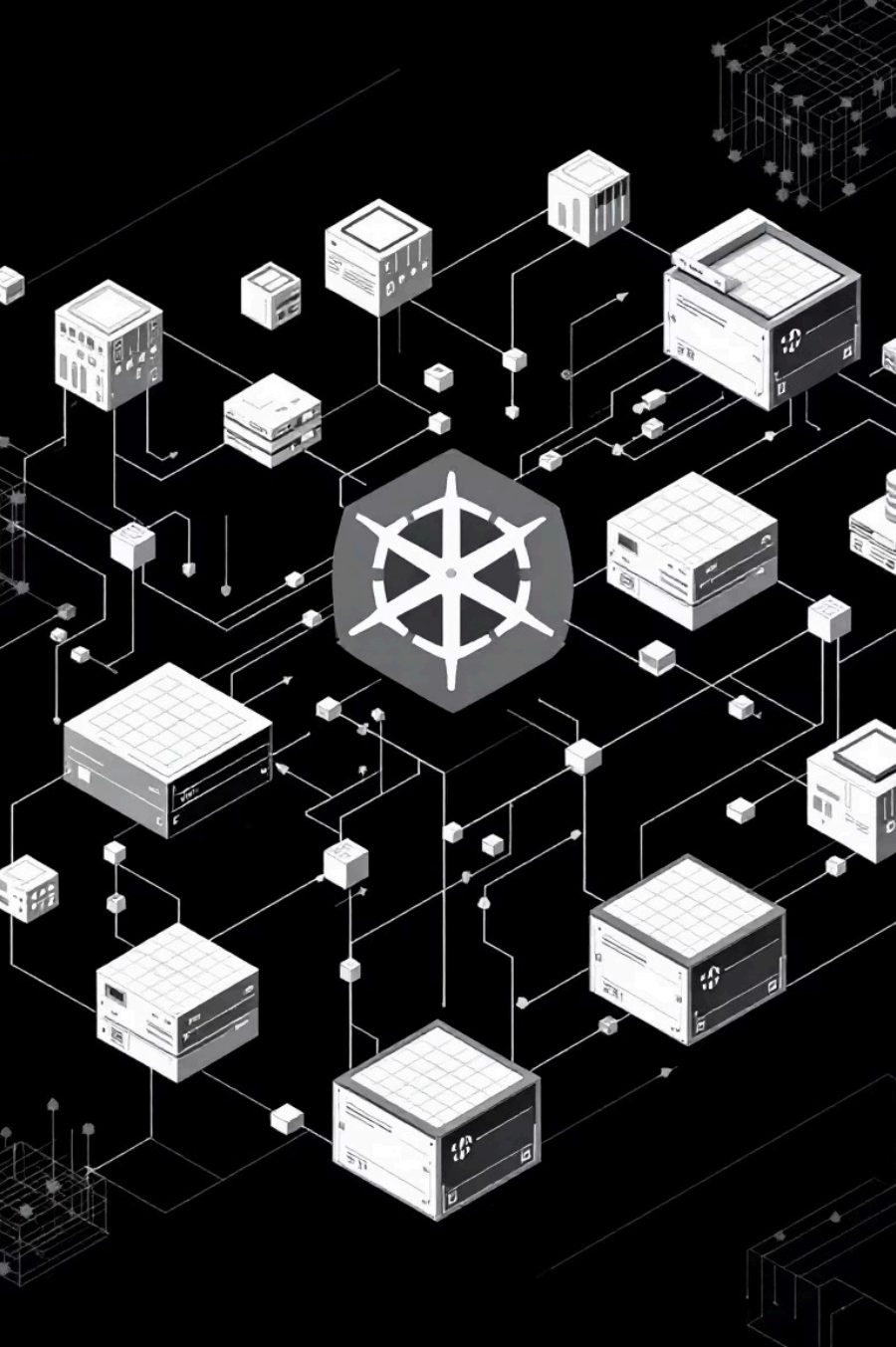




Kubernetes : Gérez vos conteneurs à grande échelle

La solution d'orchestration de conteneurs créée par Google pour déployer, mettre à l'échelle et gérer des applications conteneurisées.

Pourquoi Kubernetes ?

Applications modernes

Doivent être **rapides, scalables** et **fiables**.

Complexité

Gérer **des centaines de conteneurs** devient complexe.

Solution

Docker exécute des conteneurs.
Kubernetes les **gère automatiquement** sur plusieurs machines.

« Les applications modernes doivent évoluer rapidement, rester disponibles et fonctionner partout. Kubernetes rend cela possible. »

Docker vs Kubernetes

Fonctionnalité	Docker	Kubernetes
Objectif	Créer et exécuter des conteneurs	Orchestrer et gérer les conteneurs
Niveau d'action	Un ou plusieurs conteneurs (Docker Compose)	Des conteneurs répartis sur plusieurs machines
Scalabilité	Manuelle	Automatique
Surveillance / Reprise	Non intégrée	Intégrée

Docker est comme un **moteur** qui fait tourner une application. Kubernetes est comme un **chef d'orchestre** qui gère des centaines de services.

Pourquoi Kubernetes ?

Origine

Créé par Google en 2014 (Borg), maintenant maintenu par la CNCF (Cloud Native Computing Foundation) et donné à la communauté open-source.

Aujourd'hui le **standard mondial** de l'orchestration de conteneurs

Automatisation

Déploiement, scalabilité, redémarrage en cas de panne, découverte de services et mise en réseau.

Kubernetes vient du grec (gouvernail) pour "diriger"



Concepts clés de Kubernetes



Pod

Plus petite unité, contient un ou plusieurs conteneurs.



Node (nœud)

Machine (virtuelle ou physique) dans le cluster.



Cluster

Ensemble de nœuds gérés ensemble par le serveur.



Deployment

Détermine comment les pods sont créés et mis à jour.



Service

Rend les pods accessibles via une IP ou un nom DNS.



Namespace

Séparation logique au sein du cluster.

Kubernetes utilise des briques simples pour gérer des systèmes complexes.

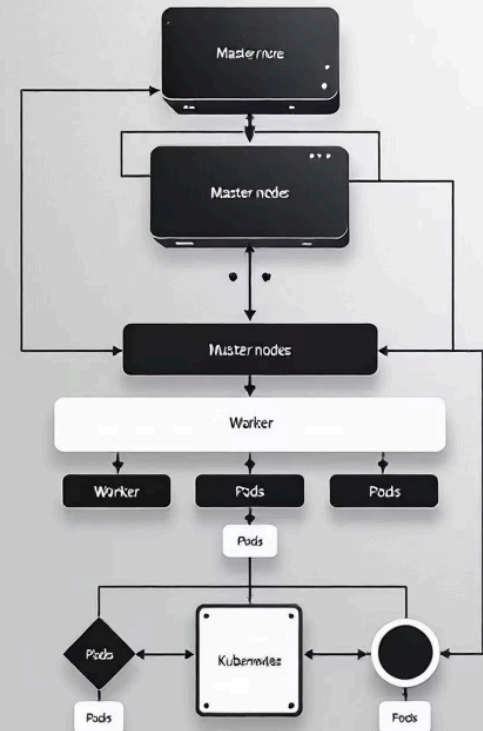
Architecture Kubernetes

Structure

- Un **cluster** contient plusieurs **nœuds** (machines)
- Chaque nœud exécute des **pods** (avec des conteneurs)

Composants

- Le **kube-apiserver** orchestre les communications
- Les **contrôleurs** assurent la cohérence
- Les **services** exposent les applications



Fonctionnalités principales

1

Auto-rétablissement

Redémarre automatiquement les conteneurs en échec.

2

Scalabilité automatique

Ajuste le nombre de pods selon la charge.

3

Mises à jour progressives

Rolling updates sans coupure de service.

4

Répartition de charge

Distribution automatique du trafic entre les pods.

Cas d'usage concrets

Applications

- **Microservices** : chaque service dans un pod indépendant
- **Applications cloud-native** : fonctionne sur AWS
- **AzureDevOps / CI-CD** : intégration avec GitLab CI, Helm

Entreprises utilisatrices



Netflix



Spotify



Doctolib

Exemple YAML + commandes

Fichier YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  template:
    spec:
      containers:
      - name: app
        image: myapp:v1
```

Commandes de base

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl describe service my-app
```





Pour aller plus loin

Documentation

- **Site officiel** (<https://kubernetes.io/>)
- **Play with Kubernetes** (<https://play-with-k8s.com/>)
- **Kube Academy** (<https://kube.academy/>)

Outils

- **Minikube** : tester Kubernetes en local
- **Helm** : gestion de paquets pour Kubernetes

Conclusion

- Kubernetes est un **orchestrateur de conteneurs**
- Il automatise la gestion de ton app en production
- Idéal pour les applications scalables, distribuées et microservices